

Universidad Nacional Experimental de Guayana

Ingeniería en Informática

Ingeniería de Software I

**INFORME DE PLANIFICACIÓN
PLAN DE 4 SEMANAS
Metodología Extreme Programming**

Proyecto: Soporte Telecom

Docente: Dubraska Roca

Estudiante: Jhordam Aguilera V-30809788

Ciudad Guayana, 4 de septiembre del 2026

1. ¿Qué es Extreme Programming (XP)?

Extreme Programming (XP) es una metodología ágil de desarrollo de software enfocada en la calidad del código y la capacidad de respuesta ante cambios de requisitos. Fue creada por Kent Beck a finales de los años 90 y se destaca por sus prácticas técnicas rigurosas.

¿Por qué XP para este proyecto?

El proyecto Soporte Telecom se beneficia de XP por las siguientes razones:

Razón	Justificación
Requisitos cambiantes	El soporte al cliente evoluciona rápidamente: nuevos canales, nuevas reglas de negocio, SLAs dinámicos. XP permite adaptarse sin perder calidad.
Calidad crítica	Un sistema de soporte con bugs puede causar pérdida de clientes. Las prácticas de XP (TDD, pair programming, CI) aseguran confiabilidad.
Integración con múltiples servicios	Supabase, n8n, Vercel, chat en tiempo real. El Continuous Integration de XP integra todo automáticamente.
Desarrollador full-stack	Un solo desarrollador con capacidad full-stack puede aplicar todas las prácticas XP de forma autónoma, manteniendo calidad y velocidad.
Feedback constante del cliente	El ciclo corto de iteraciones (1 semana) permite validar funcionalidades reales con el Product Owner frecuentemente.

2. Prácticas Clave de XP Aplicadas

XP se compone de 12 prácticas fundamentales. A continuación se detallan las que aplicamos en este proyecto:

Práctica	Descripción	Aplicación en Soporte Telecom
TDD	Escribir la prueba antes del código. Ciclo: rojo → verde → refactorizar.	Tests unitarios para hooks (useTickets, useChat), tests de integración con Supabase, tests de componentes UI.
Integración Continua	Integrar código al repositorio principal varias veces al día. Automatizar build y tests.	GitHub Actions ejecuta lint + tests en cada commit. Vercel despliega preview automáticamente.
Refactorización Continua	Mejorar la estructura del código sin cambiar su comportamiento.	Mejorar abstracciones en lib/supabase/, extraer componentes comunes en components/common/.
Small Releases	Entregar versiones pequeñas y funcionales con frecuencia.	Cada sprint entrega un incremento funcional completo, desplegado en staging.
Simple Design	El diseño más simple que funcione. No sobre-ingeniar.	Componentes React funcionales, sin patrones innecesarios. Tailwind CSS para estilos consistentes sin frameworks pesados.
Full Code Ownership	El desarrollador tiene control total y responsabilidad sobre todo el código del proyecto.	Un solo desarrollador administra toda la base de código: auth, tickets, chat, admin, workflows.
Coding Standards	Convenciones de código aplicadas de forma consistente.	TypeScript estricto, ESLint, Prettier. Convenciones de nombrado en componentes, hooks y stores.
40-Hour Week	Mantener ritmo sostenible. No horas extra habituales.	Sprints de 1 semana con carga realista. Buffer de 5 puntos para imprevistos.
On-Site Customer	El cliente o representante está disponible para el equipo.	Product Owner disponible para aclaraciones en tiempo real. Demo al final de cada sprint.
Planning Game	El cliente prioriza historias por valor de negocio. El desarrollador estima esfuerzo.	Planning al inicio de cada semana. El desarrollador estima con Fibonacci, el PO prioriza.
System Metaphor	Una historia simple que explique cómo funciona el sistema.	Soporte Telecom es el canal directo entre clientes y nuestro equipo de soporte, con chat en vivo y seguimiento de incidencias.

3. Definition of Ready (DoR)

Una historia de usuario está lista para ser trabajada cuando:

- Tiene un enunciado claro en formato "Como [rol], quiero [acción] para [beneficio]".
- Los criterios de aceptación están definidos y medibles.
- Se han identificado las dependencias técnicas.
- El desarrollador ha estimado el esfuerzo (Story Points).
- El diseño/interfaz está definido o es trivial de implementar.
- La historia es negociable (no está over-especificada).

4. Definition of Done (DoD)

Una historia de usuario está completada cuando:

- El código está escrito y pasa todas las pruebas (TDD rojo → verde → refactor).
- Tests unitarios con cobertura mínima del 80%.
- Código revisado mediante auto-revisión y verificación de criterios de aceptación.
- Integración exitosa con Supabase (Auth, RLS, Realtime).
- UI responsiva (desktop + mobile).
- Desplegado en staging (Vercel preview).
- Documentación de API actualizada (si aplica).
- Criterios de aceptación verificados por el Product Owner.
- No introduce deuda técnica nueva sin documentar.

5. Backlog del Producto

ID	Historia de Usuario	Puntos	Prioridad
US-01	Como cliente, quiero registrarme e iniciar sesión para acceder al sistema	5	Alta
US-02	Como cliente, quiero crear tickets de soporte para reportar incidencias	8	Alta
US-03	Como agente, quiero listar y filtrar tickets para gestionar mi carga	5	Alta
US-04	Como agente, quiero ver detalles de un ticket y agregar comentarios	5	Alta
US-05	Como cliente, quiero ver el historial de mis tickets	3	Media
US-06	Como agente, quiero actualizar el estado de un ticket	3	Alta
US-07	Como cliente, quiero chatear en tiempo real con un agente	13	Alta
US-08	Como agente, quiero gestionar sesiones de chat	8	Alta
US-09	Como supervisor, quiero ver el dashboard con métricas	8	Alta
US-10	Como cliente, quiero acceder a la base de conocimiento	5	Media
US-11	Como admin, quiero gestionar usuarios	8	Alta
US-12	Como admin, quiero definir categorías y prioridades	3	Media
US-13	Como sistema, quiero auto-asignar tickets a agentes	5	Media
US-14	Como sistema, quiero escalar tickets por SLA	5	Media
US-15	Como cliente, quiero recibir notificaciones por email	5	Media
US-16	Como supervisor, quiero generar reportes de rendimiento	8	Media
US-17	Como admin, quiero configurar reglas de SLA	5	Media
US-18	Como sistema, quiero un chatbot de nivel 1	13	Baja
US-19	Como agente, quiero ver métricas de desempeño personal	5	Baja
US-20	Como admin, quiero auditar acciones críticas	5	Baja

Total estimado: 120 Story Points

6. Plan Detallado por Semana

Semana 1 — Fundamentos + Auth + Tickets

Objetivo: Establecer la base del sistema con autenticación y gestión de tickets.

Story Points: 29

Día	Actividad	Práctica XP	Entregable
Lunes	Planning Game: estimar y priorizar US-01 a US-06	Planning Game	Sprint backlog definido
Lunes	Configurar CI: GitHub Actions + ESLint + tests	Integración Continua	Pipeline activo
Martes	US-01: Auth con Supabase (registro + login + roles)	TDD	Auth funcional
Miércoles	US-01: Middleware de protección de rutas	TDD	Rutas protegidas
Miércoles	US-02: Crear tickets (form + validación)	TDD + Simple Design	Formulario de tickets
Jueves	US-02: Backend de tickets (Supabase + RLS)	TDD + Full Ownership	CRUD de tickets
Jueves	US-03: Listar y filtrar tickets (vista agente)	TDD + Refactorización	Vista de tickets
Viernes	US-04: Detalle de ticket + comentarios	TDD + Refactorización	Detalle funcional
Viernes	US-05 + US-06: Historial + actualizar estado	Full Ownership	Tickets completos
Viernes	Sprint Review + Retrospective	Small Releases	Demo + mejoras identificadas

Prácticas XP activas esta semana:

- TDD: Cada funcionalidad tiene su prueba antes del código.
- Full Code Ownership: Control total sobre auth, tickets y middleware.
- CI: GitHub Actions ejecuta tests en cada commit.
- Simple Design: Componentes funcionales sin sobre-ingeniería.

Semana 2 — Chat en Tiempo Real + Dashboard

Objetivo: Implementar chat bidireccional y panel de métricas.

Story Points: 34

Día	Actividad	Práctica XP	Entregable
Lunes	Planning Game: US-07 a US-10	Planning Game	Sprint backlog
Lunes	US-07: Diseñar schema de chat (sessions + messages)	Simple Design	Schema aprobado
Martes	US-07: Sesiones de chat con Supabase Realtime	TDD	Chat funcional
Miércoles	US-07: UI de chat (cliente + agente)	TDD + Simple Design	Interface de chat
Jueves	US-08: Gestión de sesiones	TDD + Full Ownership	Gestión de chat

	(agente: aceptar/cerrar)		
Jueves	US-09: Dashboard - métricas de tickets	Refactorización	Dashboard v1
Viernes	US-09: Dashboard - métricas de usuarios y chats	TDD	Dashboard completo
Viernes	US-10: Base de conocimiento (artículos + categorías)	Simple Design	KB funcional
Viernes	Sprint Review + Retrospective	Small Releases	Demo + mejoras

Prácticas XP activas esta semana:

- TDD: Tests de integración para Supabase Realtime.
- Full Code Ownership: Control total sobre chat, dashboard y knowledge base.
- Refactorización: Optimizar queries de métricas después de implementar.
- Simple Design: Interfaces de chat limpias y funcionales.

Semana 3 — Administración + Automatización

Objetivo: Panel admin y workflows de automatización con n8n.

Story Points: 26

Día	Actividad	Práctica XP	Entregable
Lunes	Planning Game: US-11 a US-15	Planning Game	Sprint backlog
Lunes	US-11: CRUD de usuarios (admin panel)	TDD	Gestión de usuarios
Martes	US-11: Estados de usuario (active/inactive/suspended)	TDD	Estados funcionales
Miércoles	US-12: Configurar categorías y prioridades	Simple Design + TDD	Config de tickets
Miércoles	US-13: Workflow n8n auto-asignación de tickets	Integración Continua	Workflow activo
Jueves	US-14: Workflow n8n escalación por SLA	TDD + Refactorización	Escalación automática
Jueves	US-15: Workflow n8n notificaciones por email	Full Ownership	Notificaciones
Viernes	Refactorización: extraer helpers comunes	Refactorización	Código más limpio
Viernes	Sprint Review + Retrospective	Small Releases	Demo + mejoras

Prácticas XP activas esta semana:

- Integración Continua: Cada workflow de n8n se prueba con datos reales en staging.
- Full Code Ownership: Control total sobre admin, usuarios y workflows n8n.
- Simple Design: Workflows n8n con nodos claros y documentados.
- Refactorización: Extraer helpers compartidos entre admin y dashboard.

Semana 4 — Reportes + Chatbot + Calidad Final

Objetivo: Cierre del sistema con reportes, chatbot y auditoría.

Story Points: 36

Día	Actividad	Práctica XP	Entregable
Lunes	Planning Game: US-16 a US-20	Planning Game	Sprint backlog
Lunes	US-16: Módulo de reportes de rendimiento	TDD	Reportes v1
Martes	US-16: Gráficas y exportación de datos	TDD + Refactorización	Reportes completos
Miércoles	US-17: Configuración de reglas de SLA	Simple Design + TDD	SLA configurable
Miércoles	US-18: Chatbot nivel 1 (intents en n8n)	TDD	Chatbot funcional
Jueves	US-19: Métricas de desempeño personal	Full Ownership	Dashboard agente
Jueves	US-20: Sistema de auditoría	TDD	Auditoría activa
Viernes	Refactorización final + limpieza de código	Refactorización	Código production-ready
Viernes	Sprint Review FINAL + Retrospective del proyecto	Small Releases	Demo final + lecciones

Prácticas XP activas esta semana:

- Refactorización: Limpieza de código para producción.
- TDD: Tests de regresión para todo el sistema.
- Full Code Ownership: Revisión final de todas las partes críticas.
- Small Releases: Preparar versión para despliegue a producción.

7. Velocidad y Ciclo de Desarrollo XP

Semana	SP	Historias	Ciclo XP por Día
Semana 1	29	6	TDD → Code → Test → Refactor → Commit
Semana 2	34	4	TDD → Code → Test → Refactor → Commit
Semana 3	26	5	TDD → Code → Test → Refactor → Commit
Semana 4	36	5	TDD → Code → Test → Refactor → Commit

Ciclo diario de desarrollo XP (desarrollador solo):

- 1. Escribir prueba (falla) — TDD (rojo)
- 2. Escribir código mínimo (pasa) — TDD (verde)
- 3. Refactorizar — Refactor
- 4. Auto-revisión de código — Self-Review
- 5. Commit + push — CI
- 6. Integrar + build automático — CI

8. Estructura del Proyecto (Full Ownership)

El desarrollador tiene control total sobre todo el código del proyecto:

```
src/
├── app/
│   ├── (auth)/           # Login, Register → Full Ownership
│   ├── (dashboard)/      # Tickets, Chat, KB → Full Ownership
│   └── admin/            # Admin panel → Full Ownership
├── components/
│   ├── common/          # UI reusable → Full Ownership
│   ├── tickets/         # Ticket components → Full Ownership
│   ├── chat/            # Chat components → Full Ownership
│   └── admin/           # Admin components → Full Ownership
├── hooks/               # useAuth, useTickets, useChat → Full Ownership
├── stores/              # Zustand stores → Full Ownership
├── lib/
│   └── supabase/        # Client, Server, Middleware → Full Ownership
└── types/               # TypeScript types → Full Ownership

n8n-workflows/          # Workflows de automatización → Full Ownership
tests/                  # Tests unitarios e integración → Full Ownership
```

9. Pipeline de Integración Continua (Simplificado)

Trigger	Acción	Resultado
Commit a main	GitHub Actions: lint + tests	Feedback inmediato al desarrollador
Build exitoso	Vercel: build + deploy a staging	Versión preview disponible
Merge a main	Vercel: deploy automático	Incremento integrado al sistema

Herramientas de CI/CD:

- GitHub Actions: Lint, typecheck, tests unitarios e integración.
- Vercel: Despliegue automático de previews.
- Supabase: Migraciones de schema versionadas.

10. ¿Por qué XP sobre otras metodologías?

Aspecto	XP	Scrum	Kanban
Enfoque principal	Calidad técnica + prácticas de ingeniería	Proceso + gestión de trabajo	Flujo continuo de trabajo
Prácticas técnicas	TDD, Self-Review, CI, Refactoring	No prescriptivo	No prescriptivo
Duración del ciclo	1 semana (iteración corta)	2-4 sprints	Continuo
Requisitos	Ambiguos / cambiantes	Medianamente estables	Variables
Feedback del cliente	Frecuente (on-site customer)	Al final del sprint	Continuo
Mejor para	Proyectos técnicos con equipo pequeño	Proyectos con stakeholders múltiples	Mantenimiento y soporte

11. Riesgos y Mitigación

Riesgo	Impacto	Mitigación (práctica XP)
Punto único de fallo (1 desarrollador)	Alto	Documentación completa, tests exhaustivos, TDD para prevenir regresiones.
TDD alarga la fase inicial	Medio	TDD reduce bugs en producción un 40%. La inversión se recupera en estabilidad.
Sin revisión de código por pares	Medio	Auto-revisión sistemática + pipeline de CI.
Product Owner no disponible	Alto	On-site Customer requiere disponibilidad. Establecer horario fijo de consultas.
Dependencia de Supabase	Bajo	Full Code Ownership permite investigar alternativas.
Sobrecarga de trabajo	Alto	Respetar 40-hour week. Buffer de 5 puntos por sprint.

12. Conclusiones

La metodología Extreme Programming es la elección óptima para Soporte Telecom porque prioriza la calidad del código y la capacidad de adaptación, dos factores críticos en un sistema de soporte en producción.

Resumen del plan de 4 semanas:

- Semana 1: Auth + Tickets (29 SP) — Fundamentos del sistema.
- Semana 2: Chat + Dashboard (34 SP) — Comunicación en tiempo real.
- Semana 3: Admin + Automatización (26 SP) — Gestión y workflows.
- Semana 4: Reportes + Chatbot (36 SP) — Inteligencia y cierre.

Próximos pasos:

- Aprobar este plan con stakeholders.
- Configurar pipeline de CI completo.
- Iniciar Semana 1 con Planning Game.